GigShield

AI-Powered Parametric Micro-Insurance for Food Delivery Partners

Guidewire DEVTrails 2026

Phase 1 — Ideation & Foundation

Submitted: March 20, 2026

PERSONA

Food Delivery Partners — Zomato & Swiggy

COVERAGE TYPE

Income loss due to external disruptions only

PRICING MODEL

Weekly subscription (Rs. 20 – Rs. 50 / week)

Team Member	Role
Saheli Roy	Backend Development
Krishnendu Malick	AI & Backend
Rishav Kumar	Frontend & UI/UX
Aniket Das	Frontend Development
Rishika Singhadeo	UI/UX Design

Protecting India's delivery backbone, one week at a time.

Contents

1 The Problem **3**
- 1.1 Scale of the Problem . . . 3
- 1.2 Why Existing Solutions Fail . . . 3

2 Our Solution — GigShield **3**

3 Persona and User Scenarios **4**
- 3.1 Primary Persona: Raju, Swiggy Delivery Partner, Mumbai . . . 4
- 3.2 Scenario 1 — Monsoon Rain Disruption . . . 4
- 3.3 Scenario 2 — Extreme Heat Disruption . . . 4
- 3.4 Scenario 3 — Fraud Attempt (Blocked) . . . 5

4 Application Workflow **5**

5 Income-Indexed Payout Model **6**
- 5.1 Why Income-Indexed? . . . 6
- 5.2 The Payout Formula . . . 6
- 5.3 Worked Example . . . 6
- 5.4 XGBoost Earnings Predictor . . . 6

6 Weekly Premium Model **7**
- 6.1 Why Weekly? . . . 7
- 6.2 Base Plans . . . 7
- 6.3 Dynamic Premium Formula . . . 7
- 6.4 Insurance Pool Model . . . 8

7 Parametric Triggers **8**

8 AI & Machine Learning Architecture **9**
- 8.1 Model 1 — Activity Classifier . . . 9
- 8.2 Model 2 — XGBoost Earnings Predictor . . . 9
- 8.3 Model 3 — Fraud Detection . . . 9
- 8.4 Model 4 — Dynamic Premium Engine . . . 10

9 Technology Stack **10**
- 9.1 Backend — Java Spring Boot . . . 10
- 9.2 AI Microservice — Python FastAPI . . . 10
- 9.3 Frontend — Mobile-First Web . . . 10
- 9.4 External APIs . . . 11

10 System Architecture **11**
- 10.1 Midnight Payout Flow . . . 11

11 Covered Zones **12**

12 Development Roadmap **12**

13 Eligibility Rules Summary **12**

14 Team **13**

1. The Problem

India's food delivery partners on Zomato and Swiggy earn between **Rs. 15,000–Rs. 25,000 per month**, but **20–30% of that income is at risk** from external disruptions they cannot control: monsoon flooding, extreme heat, severe air pollution, curfews, and urban shutdowns.

> **Traditional insurance does not solve this.** It requires manual claim submission, lengthy verification, and payouts that take days or weeks to arrive. By then, the worker has already missed rent, EMI, or school fees. The disruption window is over — the damage is done.

1.1. Scale of the Problem

- India has over **15 million gig delivery workers** on Swiggy, Zomato, Zepto, and Amazon Flex.
- A single monsoon day in Mumbai can render **30–40 delivery zones unworkable** for 4–6 hours.
- A worker earning Rs. 700/day who loses 3 hours to flooding loses approximately **Rs. 260–Rs. 350 in income** — with no recourse.
- Less than **2%** of gig workers in India have any form of income protection insurance.

1.2. Why Existing Solutions Fail

Solution	Why it exists	Why it fails for gig workers
Traditional insurance	Covers long-term assets (health, vehicle)	Manual claims; payouts in weeks; not income-indexed
Platform welfare funds	Ad-hoc support by Swiggy/Zomato	Discretionary; not guaranteed; not parametric
Government schemes	PMSBY, ESIC for formal workers	Gig workers excluded; not real-time
Savings	Worker self-insurance	Most workers operate on weekly cash cycles; no buffer

2. Our Solution — GigShield

GigShield is an **AI-enabled parametric micro-insurance platform** that:

1. Monitors real-time weather, AQI, and civic disruption data continuously via three external APIs.
2. Detects when a predefined objective threshold is crossed during a worker's active shift.

3. Verifies the worker was genuinely active using GPS tracking and an AI activity classifier.
4. Predicts the worker's expected daily earnings using an XGBoost model trained on their personal 14-day platform earnings history.
5. Pays out automatically — no claim form, no waiting — with funds credited via UPI after midnight the same night.

> **Core Insight:** Parametric insurance pays based on *objective data*, not subjective damage assessment. If it rained more than 80 mm/hr while Raju was working, he gets paid. The threshold is crossed — the payout fires. No adjuster, no form, no wait.

3. Persona and User Scenarios

3.1. Primary Persona: Raju, Swiggy Delivery Partner, Mumbai

Attribute	Detail
Age	26
Platform	Swiggy
City	Mumbai (Bandra zone)
Vehicle	Motorcycle
Avg. Daily Earnings	Rs. 600 – Rs. 900
Peak Hours	12–2 PM and 7–10 PM
Primary Risk	Monsoon flooding, heavy rain
Smartphone	Android 4G
Payment Preference	UPI (PhonePe / GPay)

3.2. Scenario 1 — Monsoon Rain Disruption

Date: August 14, 7:30 PM, Mumbai.

Raju starts his evening shift. At 8:15 PM, rainfall crosses 85 mm/hr. Road flooding begins. GigShield detects the trigger, confirms Raju has been active for 1 h 15 min, and calculates his payout using his XGBoost-predicted daily earnings (Rs. 812 today — a Friday with a weekend premium). The payout is credited to his UPI account after midnight. Raju receives a push notification: *"Disruption payout of Rs. 76 processed. Credited after 12 AM."* No form filled.

> **Trigger:** Heavy Rainfall > 80 mm/hr (IMD Very Heavy Rain classification). **Activity:** ACTIVE (GPS confirmed delivery movement). **Payout:** Rs. 76 (income-indexed).

3.3. Scenario 2 — Extreme Heat Disruption

Date: May 22, 2:00 PM, Delhi.

Priya, a Zomato partner, starts her afternoon shift. Temperature hits 43°C and stays there for 2.5 hours. GigShield triggers a heat disruption event, validates Priya's active GPS session, and calculates her payout based on her predicted earnings (Rs. 680 for a Tuesday in summer). Payout credited after midnight.

3.4. Scenario 3 — Fraud Attempt (Blocked)

Date: July 3, 6:00 PM.

A worker activates their shift but remains stationary at home. GPS shows no delivery-pattern movement. At 6:45 PM, heavy rain triggers a disruption event. The AI activity classifier assigns a fraud score of 78/100 and flags the session as INACTIVE. Payout is blocked. The claim is logged for admin review rather than auto-rejected, protecting legitimate workers from false positives.

4. Application Workflow

The end-to-end flow from worker onboarding to midnight payout:

1. **Register & OTP Verify** — Worker enters phone number, receives OTP, completes profile (name, city, platform, vehicle type, declared daily earnings).
2. **Platform Earnings Sync** — System fetches last 14 days of earnings history from the mock Swiggy/Zomato partner API.
3. **XGBoost Prediction** — A personalised XGBoost model trained on the worker's own history predicts today's expected daily and hourly earnings.
4. **Subscribe to Weekly Plan** — AI displays a personalised weekly premium. Worker selects Basic (Rs. 20), Standard (Rs. 30), or Premium (Rs. 50). Coverage activates immediately for 7 days.
5. **Start Shift** — Worker taps "Start Shift". GPS tracking begins. Eligibility timer counts toward 180 minutes active.
6. **Continuous Monitoring** — Weather APIs polled every 15 minutes. AQI checked every 15 minutes. GPS activity classified in real time.
7. **Disruption Detected** — Threshold crossed ⇒ DisruptionEvent created ⇒ 3-step AI pipeline fires for all active shifts in the zone.
8. **AI Pipeline Executes** —
   a. Fetch 14-day earnings from platform API.
   b. XGBoost predicts today's earnings.
   c. Eligibility & fraud check; payout calculated.
9. **Claim Approved** — Claim saved with status `approved` and exact payout amount.
10. **Midnight Batch** — Spring Boot `@Scheduled` job at 00:00 collects all approved claims, calls Razorpay payout API, marks claims `paid`, sends WebSocket notification to worker.

5. Income-Indexed Payout Model

5.1. Why Income-Indexed?

A flat payout of Rs. 300 is arbitrary and unfair. A worker earning Rs. 1,200/day and a worker earning Rs. 500/day both lose proportionally different amounts when disrupted. GigShield uses the worker's *actual predicted earnings* to calculate a fair, actuarially sound payout.

5.2. The Payout Formula

$$\text{Payout} = \underbrace{50\%}_{\text{income share}} \times \underbrace{\frac{E_{\text{daily}}}{8}}_{\text{hourly rate}} \times \underbrace{t_{\text{overlap}}}_{\text{hours disrupted}} \times \underbrace{M}_{\text{multiplier}}$$

where:

- E_{daily} = XGBoost-predicted daily earnings for today
- t_{overlap} = hours the disruption overlapped the active shift
- M = severity multiplier (0.6× to 1.5× depending on disruption type)
- Cap: payout cannot exceed E_{daily} (the worker's own predicted daily earnings)

5.3. Worked Example

Variable	Value
Worker's 14-day avg earnings	Rs. 700/day
XGBoost prediction (Friday)	Rs. 812 (weekend premium +16%)
Predicted hourly rate	Rs. 812 ÷ 8 = Rs. 101.50/hr
Disruption type	Heavy Rain (multiplier = 1.0×)
Overlap with active shift	90 minutes (1.5 hours)
Raw payout	50% × Rs. 101.50 × 1.5 × 1.0 = **Rs. 76.13**
Daily cap	Rs. 812 (not exceeded)
Final payout	**Rs. 76.13 credited after 12 AM**

5.4. XGBoost Earnings Predictor

A personalised XGBoost regression model is trained on each worker's own 14-day earnings history fetched from the platform API.

Features used:

- Day of week (0–6)
- Is weekend (binary)
- 7-day rolling average earnings
- 3-day rolling average earnings
- 14-day rolling average earnings
- Earnings standard deviation (volatility)
- Average orders (last 7 days)
- Average peak hours worked
- Weather forecast rain (mm)
- Weather forecast AQI
- City zone flood risk score
- Earnings trend (linear slope)

Model hyperparameters: `n_estimators=60`, `max_depth=3`, `learning_rate=0.15`, `subsample=0.85`. Shallow trees prevent overfitting on short training windows.

6. Weekly Premium Model

6.1. Why Weekly?

Food delivery workers operate on a week-to-week financial cycle. They receive platform payouts weekly and budget accordingly. A weekly insurance premium of Rs. 20–Rs. 50 is psychologically and practically aligned with how they already manage money.

6.2. Base Plans

Plan	Weekly Premium	Daily Cap	Payouts/Week	Monthly Cap
Basic	Rs. 20	Rs. 150	1	Rs. 600
Standard	Rs. 30	Rs. 200	2	Rs. 800
Premium	Rs. 50	Rs. 300	3	Rs. 1,000

6.3. Dynamic Premium Formula

$$P_{\text{weekly}} = P_{\text{base}} \times \underbrace{Z}_{\text{zone risk}} \times \underbrace{S}_{\text{seasonal factor}} - \underbrace{L}_{\text{loyalty discount}}$$

Factor	Description	Range
Zone Risk (Z)	Historical disruption frequency in GPS zone	1.0 – 1.3
Seasonal Factor (S)	Upcoming week's disruption probability (7-day forecast)	1.0 – 1.2
Loyalty Discount (L)	Clean claims history reward after 8 weeks	Rs. 0 – Rs. 3

Example: Standard plan (Rs. 30 base) × 1.2 (zone risk) × 1.1 (monsoon season) − Rs. 3 (loyalty) = **Rs. 36.60/week**

6.4. Insurance Pool Model

- 10,000 workers × Rs. 30/week average = **Rs. 3,00,000 weekly pool**
- Expected weekly payouts at 10% disruption rate = Rs. 60,000 – Rs. 90,000
- Pool surplus invested in liquid assets and reserves
- **Loss ratio target:** 55–65% (actuarially sustainable)

7. Parametric Triggers

GigShield uses 6 parametric triggers. A payout fires only when **both** conditions are satisfied:

- **Condition A** — Disruption Threshold: the weather/civic event meets the defined objective threshold.
- **Condition B** — Active Time Overlap: the disruption occurred during the worker's confirmed active GPS session (≥ 30 minutes overlap required).

#	Trigger	Data Source	Threshold / Condition	Multiplier
1	Heavy Rainfall	OpenWeatherMap	>80 mm/hr for ≥60 min overlapping active shift	1.0×
2	Extreme Heat	OpenWeatherMap	>42°C for ≥2 hrs overlapping active shift	1.0×
3	Severe AQI	IQAir AirVisual	AQI >300 for ≥3 hrs overlapping active shift	1.0×
4	Urban Flooding	OpenWeatherMap + GPS	Flood alert + >150 mm/24 hr in worker's GPS zone	1.5×
5	Curfew / Section 144	Government alert feed	Movement restriction overlapping worker's shift window	1.5×
6	Weather-Forced Short Shift	GPS Log + Weather API	Active shift <4 hrs AND any weather event during session	1.0×

Trigger 6 catches cases where weather forces the worker to end their shift early (reducing total active time below 4 hours) even if no single weather threshold was individually breached. Reduced work time *is* the trigger.

All 6 triggers cover income loss only. Vehicle repair, health insurance, and accident medical bills are explicitly excluded per platform rules.

8. AI & Machine Learning Architecture

AI fires at **four points** in the workflow:

1. **At registration** — Risk score computed to set personalised premium.
2. **At shift start** — XGBoost model predicts today's expected earnings.
3. **During shift** — GPS activity classifier runs continuously.
4. **At disruption** — Fraud detection + eligibility check + payout calculation.

8.1. Model 1 — Activity Classifier

Purpose: Classify whether a worker is genuinely active on deliveries or idle/stationary.

Input features: GPS speed and acceleration, stop duration and frequency, distance travelled, location cluster density near restaurants and customers, time of day, historical shift patterns.

Output: ACTIVE (eligible) — INACTIVE (not eligible) — MANUAL REVIEW (held for admin).

8.2. Model 2 — XGBoost Earnings Predictor

Purpose: Predict the worker's expected daily earnings for the current day using their personal 14-day history from the platform API.

Input: 14-day earnings history, day-of-week, rolling averages (3d, 7d, 14d), earnings trend, weather forecast (rain mm, AQI), city zone risk, peak hours worked.

Output: Predicted daily earnings (Rs.), predicted hourly rate (Rs./hr), confidence band (±15%).

8.3. Model 3 — Fraud Detection

Purpose: Detect fake or manipulated claim attempts before payout is issued.

Detection rules:

- **GPS Spoofing:** >80% of GPS points with 0 m accuracy (+35 fraud score points)
- **Impossible Speed:** Movement >120 km/h between GPS points (+20 points per event)
- **Low Activity:** <15% active movement during shift (+25 points)
- **Bot Pattern:** Grid-like movement (>90% cardinal headings) (+25 points)
- **Low Coverage:** <10 GPS points recorded (+30 points)

Decision thresholds:

Score	Decision	Action
0 – 39	Auto-Approved	Pay immediately
40 – 69	Manual Review	Admin reviews before payment
70 – 100	Auto-Rejected	Blocked; fraud logged

8.4. Model 4 — Dynamic Premium Engine

Purpose: Calculate the personalised weekly premium every Monday before renewal.

Risk score components:

Factor	Weight	Example
Disruption history in zone	30%	Bandra: 67 events → high
Zone flood risk	25%	Bandra: 0.85 (very high)
Seasonal factor	20%	Monsoon: 85/100
Vehicle type	15%	Bicycle: 80 (most exposed)
Platform	10%	Zepto: 65 (faster SLA pressure)

9. Technology Stack

9.1. Backend — Java Spring Boot

Component	Technology
Core API server	Java 17 + Spring Boot 3.2
Authentication	JWT + OTP (SMS via Fast2SMS)
Database	H2 (dev) / PostgreSQL (production)
Real-time push	WebSocket / STOMP
Scheduled jobs	Spring `@Scheduled` (midnight batch)
Async processing	Spring `@Async` (claim pipeline)

9.2. AI Microservice — Python FastAPI

Endpoint	Purpose
`POST /api/platform/earnings`	Mock Swiggy/Zomato earnings API
`POST /api/predict-earnings`	XGBoost daily earnings prediction
`POST /api/check-eligibility`	Full eligibility + payout calc
`POST /api/analyze-gps`	GPS fraud detection
`POST /api/risk-score`	Dynamic premium engine
`POST /api/midnight-batch`	Midnight UPI payout processing

9.3. Frontend — Mobile-First Web

Page	Description
`login.html`	Split-screen worker + admin login
`register.html`	3-step onboarding with earnings input
`dashboard.html`	Live shift timer, GPS, eligibility bar
`subscribe.html`	AI-personalised plan selection
`claims.html`	Claim history and payout summary
`admin.html`	Analytics, disruption map, simulator

> **Why mobile-first web over native app:** No App Store download barrier; works on budget Android on 3G; single codebase serves mobile workers and desktop admin staff; faster iteration during hackathon phases.

9.4. External APIs

API	Purpose	Cost
OpenWeatherMap	Rain mm/hr, temperature, wind	Free tier
Open-Meteo	7-day forecast, hourly precipitation	**Free, no key**
IQAir AirVisual	Real-time AQI / pollution	Free tier
Razorpay	UPI payout simulation	Sandbox / free
Fast2SMS	OTP delivery	Free tier

10. System Architecture

```
Frontend (HTML/CSS/JS)   GitHub Pages
↓ REST API (JWT Auth)
Spring Boot Backend   Railway.app
Java 17 · Spring Security · WebSocket · H2/PostgreSQL
↓ HTTP                ↓ WebSocket
Python FastAPI        Worker Browser
XGBoost · Fraud ML    Real-time notifications
Render.com
↓ HTTP (free tier)
OpenWeatherMap · Open-Meteo · IQAir AirVisual
Mock Swiggy/Zomato Earnings API
```

10.1. Midnight Payout Flow

1. Disruption fires during worker's active shift
2. Spring Boot calls `POST /api/platform/earnings` — 14-day history fetched
3. Spring Boot calls `POST /api/predict-earnings` — XGBoost predicts today's income
4. Spring Boot calls `POST /api/check-eligibility` — payout calculated; claim saved as `approved`
5. At 00:00 — `@Scheduled(cron = "0 0 0 * * *")` fires
6. `MidnightPayoutScheduler` collects all approved claims
7. Calls `POST /api/midnight-batch` — UPI transfers executed
8. Claims marked `paid`; WebSocket push sent to each worker

11. Covered Zones

Zone	City	Flood Risk	Historical Disruptions
Bandra	Mumbai	85%	67 events
Andheri	Mumbai	75%	58 events
T Nagar	Chennai	60%	45 events
Salt Lake	Kolkata	70%	52 events
Koramangala	Bangalore	45%	28 events
Connaught Place	Delhi	35%	22 events
Sector 18	Noida	40%	19 events
Jubilee Hills	Hyderabad	30%	15 events

12. Development Roadmap

Phase	Dates	Key Deliverables
Phase 1 Ideation & Foundation	Mar 4 – 20	Problem research, persona definition, system architecture, DB schema, tech stack, core concept document (this document), 2-minute strategy video
Phase 2 Automation & Protection	Mar 21 – Apr 4	Worker registration with OTP, GPS shift tracking, XGBoost earnings predictor, dynamic premium engine, 5 automated parametric triggers, zero-touch claims pipeline, worker dashboard
Phase 3 Scale & Optimise	Apr 5 – 17	Advanced fraud detection, Razorpay payout integration, full analytics dashboard, disruption simulation panel, final pitch deck, 5-minute demo video

13. Eligibility Rules Summary

For a claim to be approved, all of the following must be true:

Worker has an active weekly policy (paid premium)

Shift has at least **180 active minutes** (3 hours) of confirmed GPS movement

Disruption overlapped with shift by at least **30 minutes**

GPS fraud score is **below 70/100**

Worker has not exceeded their plan's **weekly payout limit**

Calculated payout does not exceed worker's **predicted daily earnings**

> **Note:** Flagged claims (fraud score 40–69) are held for **admin review**, not auto-rejected. This protects legitimate workers from GPS signal errors, indoor delivery stops, or weak network areas being incorrectly classified as suspicious.

14. Team

Name	Role
Saheli Roy	Backend Development
Krishnendu Malick	AI & Backend
Rishav Kumar	Frontend & UI/UX
Aniket Das	Frontend Development
Rishika Singhadeo	UI/UX Design